**Master 1 Informatique**

# Rapport : Projet SDA

## Route planning : Répondre vite à des requêtes de plus court chemin.


**Membre du groupe :**

- HARKATI Rayane
- ZAID Marouane
- MIMOUNI Saad

**Sommaire :**

1. Objectif du projet.
2. Création de la structure de données.
3. Extraction de données (OSM vers CSR).
4. Les algorithmes.
5. Les résultats.
6. Analyse des résultats.
7. Conclusion.

**Objectif du projet :**

Charger un vrai réseau routier, et répondre rapidement à des questions du type quel est le chemin le plus court de A vers B ?

Le problème c'est que sur une grande carte, l'algorithme classique Dijkstra devient trop lent si on enchaîne beaucoup de requêtes.

Donc on va implémenter et comparer 4 méthodes : **Dijkstra** (la référence), **A*** (Dijkstra + heuristique), **ALT** (**A*** amélioré avec des points repères), et **Contraction Hierarchies** (la méthode utilisée dans les vrais GPS).

Pour chaque méthode on mesure le **temps**, la **mémoire** et le **nombre de nœuds explorés** — et on compare pour voir laquelle est la plus rapide et pourquoi.

**Langage utilisé :** Python

**Bibliothèque utilisé :**Pickle, osmnx, nympy, heapq, math, time pandas, matplotlib.

**Architecture du projet:**

```
Projet_SDA/
├── data/
│   ├── extract_osm.py        # extraire graphe depuis OSM
│   ├── generate_synthetic.py  # graphe grille de test
│   ├── graph_paris.pkl       # graphe sérialisé
│   └── queries.csv           # lot de 1000 paires (s,t)
├── graph/
│   ├── __init__.py
│   ├── csr_graph.py          # structure CSR numpy
│   ├── loader.py             # parsing + chargement
│   └── haversine.py          # distance à vol d'oiseau
├── algorithms/
│   ├── __init__.py
│   ├── dijkstra.py           # baseline référence
│   ├── astar.py              # A* + heuristique
│   ├── alt.py                # ALT + landmarks
│   └── ch.py                 # Contraction Hierarchies
├── landmarks/
```

```
│   ├── selection.py        # choisir les landmarks
│   ├── precompute.py       # Dijkstra depuis chaque landmark
│   └── distances.npy       # matrice précalculée
├── benchmark/
│   ├── runner.py           # lancer tous les algos
│   ├── metrics.py          # temps, nœuds, mémoire
│   └── plots.py            # générer les figures
├── results/
├── main.py                 # point d'entrée principal
├── README.md
└── requirements.txt
```

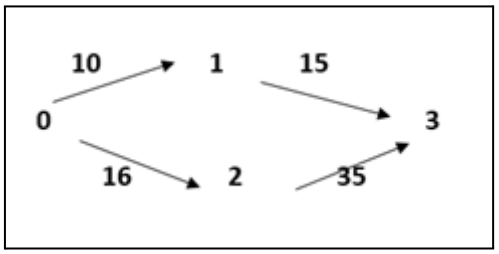
**<u>Création de la structure de données :</u>**

**<u>graph/csr_graph.py :</u>**

Le format CSR stocke tous les 3 tableaux plats numpy et c'est 10 fois moins de mémoire qu'un dictionnaire

Sur paris y'en a 300.000 nœuds

Exemple avec 4 nœuds :



Les voisins de 0

| Nœuds | 0 | 1 | 2 | 3 | |
|---|---|---|---|---|---|
| Indices( les voisins) | 1 | 2 | 3 | 3 | |
| indptr | **0 (indice 0 dans le tableau indice)** Les voisins du premier nœuds (0) commencent a partir de l'indice 0) | **2 (indice 2 dans le tableau indice)** Les voisins du deuxieme nœuds (1) commencent a partir de l'indice 2) | 3 | 4 | 4 |

**Traduction en CSR par 3 tableaux distinct :**

Nœuds[] : représente les nœuds

Indices[] : représente les voisins de chaque nœud

Indptr[] : représente l'indice ou commence chaque voisin du nœuds

**csr_graph.py :** La structure de données centrale

On définit la classe CSRGraph : une représentation du graphe routier en format CSR

Compressed Sparse Row (CSR).

- Au lieu d'une liste d'adjacence classique, tout est stocké dans 3 tableaux numpy :
- indptr : pour chaque nœud i, les arêtes qui partent de lui sont entre indptr[i] et indptr[i+1]
- indices : les nœuds destination de chaque arête
- weights : les poids (distances en mètres)
- coords : tableau optionnel (lat, lon) pour chaque nœud
- get_neighbors(node) → retourne les voisins + poids d'un nœud (utilisé par Dijkstra, A*, ALT)
- get_coords(node) → retourne la position GPS d'un nœud (utilisé par l'heuristique haversine)

**/Data :**

extract_osm.py — Graphe réel (Paris)

Télécharge le vrai réseau routier de Paris depuis OpenStreetMap via osmnx et le convertit en CSRGraph.

Les étapes :

1. Télécharge le graphe routier (network_type="drive")

2. Remappe les IDs OSM (qui sont de grands entiers arbitraires) vers 0..n-1 pour pouvoir les utiliser comme indices de

tableaux

3. Extrait les coordonnées GPS (lat, lon) de chaque nœud

4. Construit les listes d'adjacence avec les longueurs des rues en mètres

5. Convertit en format CSR et sauvegarde dans graph_paris.pkl

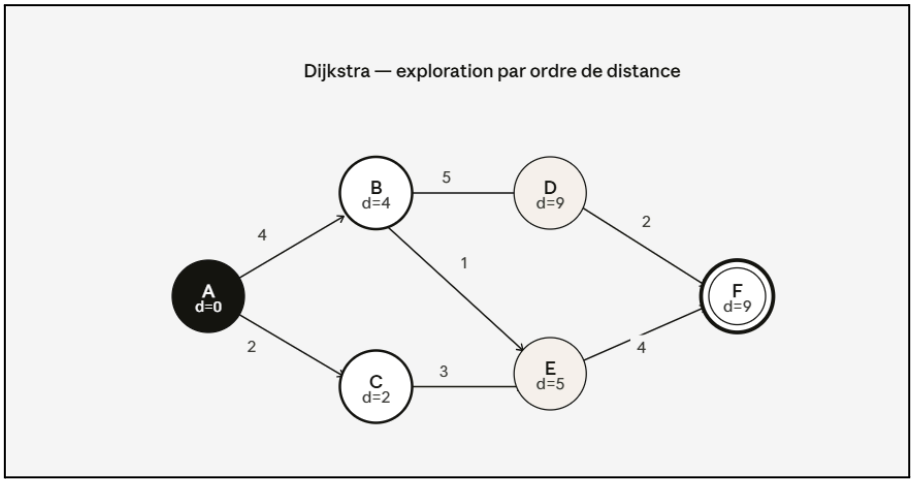
**Extraction de données (OSM vers CSR):**

**data/extract_osm.py :**

1. Utilise la bibliothèque **Pickle**.
2. Utilise la bibliothèque **osmnx**.
3. Contient une fonction **extract_osm** elle prends en paramètres deux entrées : place_name, output_path, elle retourne graph_paris en format PKL ( pkl sérialise un tuple de deux tableaux NumPy )

**Les algorithmes :**

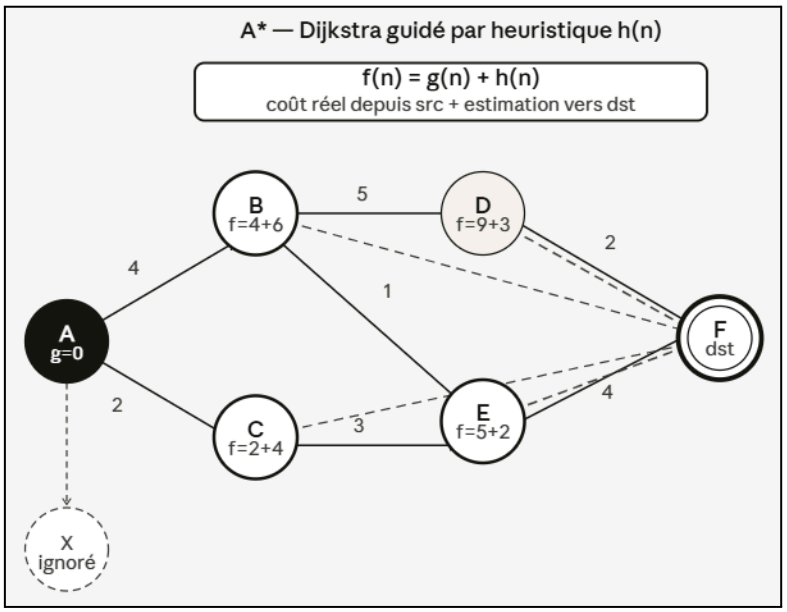
**Dijkstra.py**




Ordre d'exploration (min-heap)
1. Extraire A (d=0) → relaxer B(4), C(2)
2. Extraire C (d=2) → relaxer E(2+3=5)
3. Extraire B (d=4) → relaxer D(4+5=9), E(4+1=5)

4. Extraire E (d=5) → relaxer F(5+4=9)
5. Extraire D (d=9) → F(9+2=11 > 9, ignoré)

**a_star**



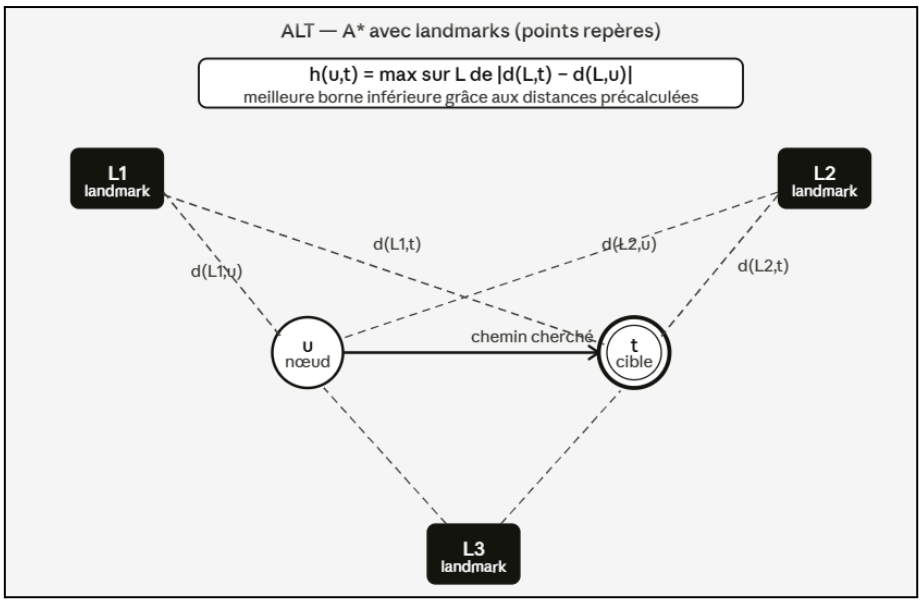

**f(n) = g(n) + h(n)** coût réel depuis src + estimation vers dst

**g(n)** = distance réelle déjà parcourue depuis la source
**h(n)** = estimation à vol d'oiseau (haversine) jusqu'à la destination
**A*** priorise **f(n)=g+h** → évite les mauvaises directions (nœud X ignoré)
--- = heuristique haversine (jamais supérieure à la vraie distance)

**L'heuristique haversine :**Sur un réseau routier, chaque nœud a des coordonnées GPS (latitude, longitude). On utilise la **distance haversine** — la distance à vol d'oiseau entre deux points — comme estimation de la distance restante.

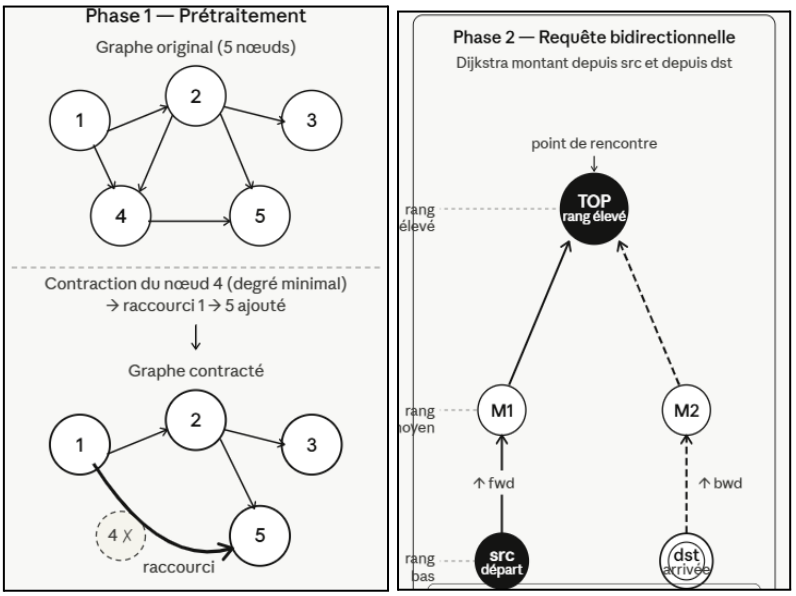
Cette heuristique est **admissible** : elle ne surestime jamais la vraie distance, car la ligne droite est toujours plus courte que n'importe quel chemin routier. Cela garantit qu'A* trouve toujours le chemin optimal.

**alt.py**




Prétraitement : Dijkstra depuis chaque landmark → matrice de distances stockée
Requête : **h(u,t)** = max|d(L,t)−d(L,u)| → borne plus serrée que haversine seule
Plus de landmarks = meilleure heuristique = moins de nœuds explorés

**CH (Contraction Hierarchies)** : on accepte de faire un **long prétraitement** une seule fois, pour que chaque requête soit ensuite **ultra rapide**



**Les 3 étapes de CH**

- **Ordonner les nœuds :**

On attribue un rang à chaque nœud — du moins important au plus important. Un nœud peu connecté (petite rue) a un rang bas, un nœud très connecté (grande avenue, autoroute) a un rang élevé.

- **Contracter les nœuds + ajouter des raccourcis :**

On contracte les nœuds du rang le plus bas au plus élevé. À chaque contraction, si supprimer un nœud allongerait un chemin, on ajoute un raccourci direct entre ses voisins.

- **Requête bidirectionnelle :**

Pour répondre à une requête s→t, on lance deux Dijkstra simultanément : un depuis s qui monte dans la hiérarchie, et un depuis t qui monte aussi. On s'arrête quand les deux se rejoignent au sommet.

**<u>Les résultats :</u>**

**On obtient des figures et un fichier CSV contenant les données**

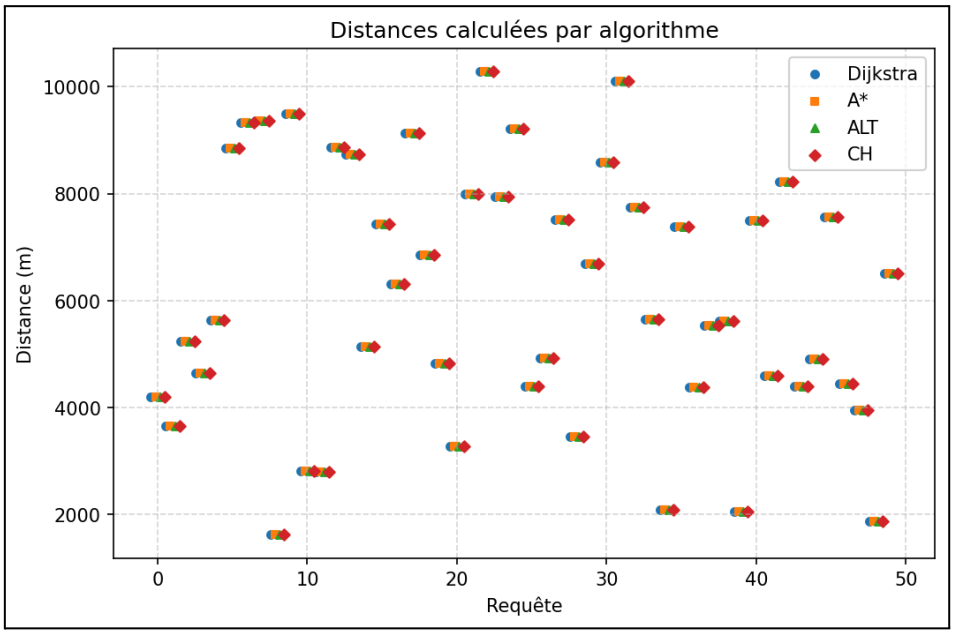



**Figure1: Distances calculée par chaque algorithme**

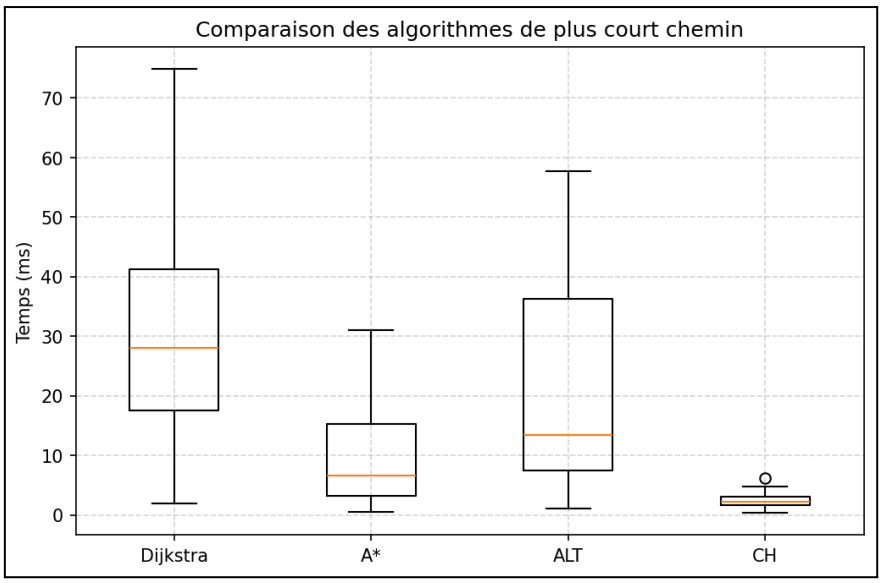



**Figure2: Comparaison des algorithmes de plus court chemin avec les données synthétiques.**

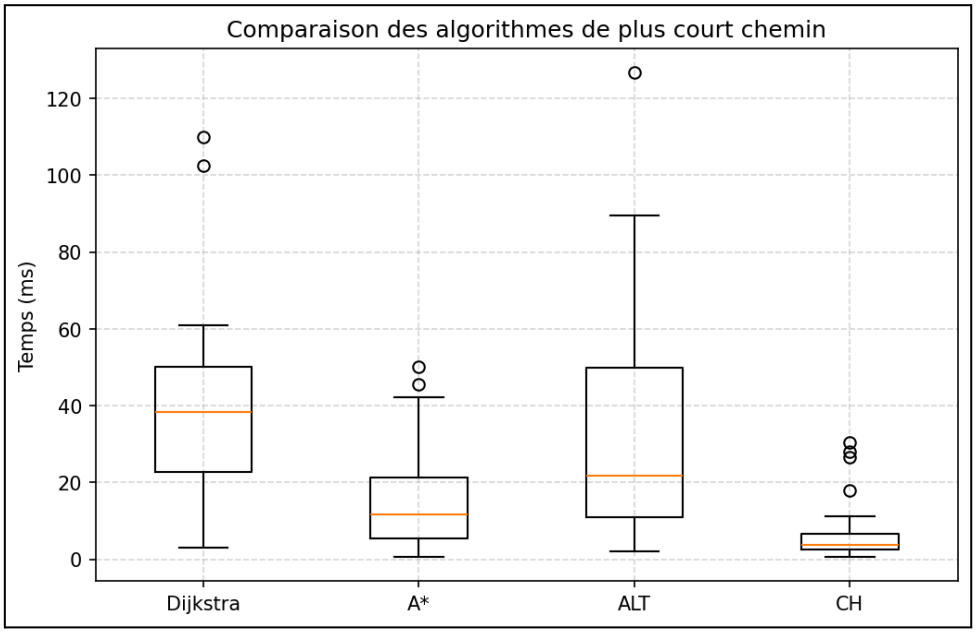



**Figure 3: Comparaison des algorithmes de plus court chemin avec les données OSM(Réelles).**

**<u>Analyse des résultats:</u>**

Le premier graphique compare les temps d'exécution des 4 algorithmes sur un lot de requêtes. **<u>“</u>**Temps d'exécution (figure 2 et 3)”

**CH** est le plus rapide — sa médiane est quasi nulle et sa boîte est très resserrée, ce qui signifie qu'il répond de manière extrêmement rapide et stable sur toutes les requêtes.

**A*** est légèrement plus rapide que Dijkstra en médiane, ce qui confirme que l'heuristique haversine aide bien à réduire l'exploration.

**ALT** affiche des temps plus élevés que prévu avec une variance importante — cela s'explique probablement par le coût du prétraitement des landmarks insuffisant pour bien guider la recherche.

**Dijkstra** présente le plus grand outlier (20ms) ce qui illustre bien son comportement sur les requêtes longues distances où il explore énormément de nœuds.

**Distances calculées (figure 1)**

Le graphique montre les distances retournées par les 4 algorithmes sur 50 requêtes.

Les points Dijkstra, A*, ALT et CH sont systématiquement superposés pour chaque requête, ce qui confirme que tous les algorithmes trouvent la même distance optimale.

Les distances varient entre 1600m et 10 300m selon les paires testées, couvrant ainsi des requêtes courtes et longues distances.

Ce résultat valide l'ensemble de toutes les implémentations.

**Remarque**: La légère visibilité du losange rouge (CH) par rapport aux autres marqueurs est uniquement due à sa taille graphique, et non à un écart de valeur.

---

**Analyse de données CSV:**

Prenons exemple (On trie par source 51 et la cible 12) :

**Données synthétiques:**

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | algorithm | query | source | target | time_ms | distance | |
| 10 | Dijkstra | 8 | 51 | 12 | 1.2932 | 7345.864405435638 | |
| 60 | A* | 8 | 51 | 12 | 0.2438 | 7345.864405435638 | |
| 110 | ALT | 8 | 51 | 12 | 0.8023 | 7345.864405435638 | |
| 160 | CH | 8 | 51 | 12 | 0.1773 | 7345.864405435639 | |
| 202 | | | | | | | |

**Données réelles :**

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | algorithm | query | source | target | time_ms | distance | |
| 28 | Dijkstra | 26 | 3431 | 4401 | 14.8525 | 4930.406729472506 | |
| 78 | A* | 26 | 3431 | 4401 | 2.7804 | 4930.406729472506 | |
| 128 | ALT | 26 | 3431 | 4401 | 11.1317 | 4930.406729472506 | |
| 178 | CH | 26 | 3431 | 4401 | 0.8774 | 4930.406729472508 | |

*Dijkstra est largement (28 fois) plus lent sur cette requête parce qu'il explore le graphe dans toutes les directions sans savoir où est la destination. A*, ALT et CH utilisent tous de l'information supplémentaire pour guider la recherche — c'est exactement ça qu'on voulait montrer dans ce projet.

**Complexité d'algorithmes:**

| Algorithme | Prétraitement | Par requête |
|---|---|---|
| Dijkstra | Aucun | O((V+E) log V) |
| A* | Aucun | O(L log V) |
| ALT | O(k×(V+E) log V) | O(L log V) |
| CH | O(V log V × d) | O(√V log V) |

**Conclusion :**

Les résultats obtenus confirment que plus on investit en prétraitement, plus les requêtes sont rapides.

CH domine avec 3.5ms par requête contre 62ms pour Dijkstra, validant ainsi l'intérêt des Contraction Hierarchies sur un vrai réseau routier.

A* et ALT offrent un bon compromis intermédiaire sans prétraitement lourd.